

COMP 3106 A – Fall 2026 – Assignment-1

1 Introduction

The assignment is an opportunity to demonstrate your knowledge on informed graph search and practice applying it to a problem.

The assignment may be completed individually, or it may be completed in small groups of two or three students. The expectations will not depend on group size (i.e. same expectations for all group sizes).

Assignment due date: 2026-10-09

Assignments are to be submitted electronically through Brightspace. It is your responsibility to ensure that your assignment is submitted properly. Copying of assignments is NOT allowed. Discussion of assignment work with others is acceptable but each individual or small group are expected to do the work themselves.

1.1 General Instructions

The assignment should contain two components: an implementation and answers to the questions below. Each component requires submission of single file. So for this assignment a complete submission will include 2 (two) files, one per component.

Implementation

Programming language: Python 3

You may use the Python Standard Library (<https://docs.python.org/3/library/>). You may also use the NumPy, Pandas, and SciPy packages (and any packages they directly depend on). Use of any additional packages requires approval of the instructor.

You must implement your code yourself. Do not copy-and-paste code from other sources, but you may use any pseudo-code we wrote in class as a basis for your implementation. Your implementation must follow the outlined specifications. Implementations which do not follow the specifications may receive a grade of zero. Please make sure your code is readable, as it will also be assessed for correctness. You do not need to prove correctness of your implementation.

You may be provided with a set of examples to test your implementation. Note that the provided examples do not necessarily represent a complete set of test cases. Your implementation may be evaluated on a different set of test cases.

The implementation will be graded both on content and use of good programming practices.

Submit the implementation as a single PY file.

Questions Section

You must answer all questions posed in Section 3 "Questions Section" below. Ensure you answers are clear and concise.

If the assignment was completed in a small group of students, you must also include a statement of contributions. This statement should identify: (1) whether each group member made significant contribution, (2) whether each group member made an approximately equal contribution, and (3) exactly which aspects of the assignment each group member contributed to.

Submit your answers to the questions below as a single PDF file.

2 Implementation

General problem description: Consider the problem of finding a path from a start position to an end position in a grid-like environment. Implement an algorithm using A* search to find the optimal path to a goal .

Agent: In this problem, assume we have an agent that starts at a start state on an $m \times n$ rectangular grid. The agent tries to find a path from the start state to a goal state (there may be multiple goal states) by moving to adjacent horizontal or vertical tiles on the grid (cannot move diagonally). The agent must collect treasures along the way. Assume the environment is fully observable; assume there always exists a path from the start state to a goal state.

Environment: An $m \times n$ grid of tiles. Some tiles are "walls". An agent cannot traverse through (or into) walls. Some tiles contain "treasure". Treasures can have integer value between 1 and 5. A treasure is "collected", when an agent visits/traverses the treasure tile. A treasure can only be collected once, so that after an agent visits a treasure tile, that tile turns into a regular tile.

In general, a tile can be of the following types:

1. The start tile (indicated by the letter "S").
2. A goal tile (indicated by the letter "G").
3. A wall (indicated by the letter "X").
4. A treasure (indicated by the integer 1, 2, 3, 4, or 5).
5. A regular tile (indicated by the integer 0).

In all cases, the cost of moving to an adjacent tile is 1(one).

Performance measure: The agent must find a path that has the smallest cost possible, while satisfying the following two conditions:

- The path begins at the start tile and ends in a goal tile
- Following the path the agent would collect treasures with the **total** value of at least 5.

Technical details: Your implementation must contain a single file named "assignment1.py" with a function named "pathfinding", which takes one input argument (you may have other variables/functions/classes in your file). The input argument should be the full file path to a comma separated value (CSV) file that contains the input grid.

The CSV file will contain a grid describing the environment. The CSV will contain a rectangular grid with one S (indicating the start tile), one or more G (indicating goal tiles), zero or more X (indicating walls), one or more integers (indicating treasures), and zero or more 0 (indicating regular tiles).

Your function should return three values.

1. The first returned value should be a list of tuples indicating the coordinates of the tiles occupied along the optimal path, in order. Assume the first index indicates the row and the second index indicates the column. Both row and column indices are integers and start at zero.
2. The second returned value should be the cost of the optimal path.
3. The third returned value should be the number of states explored during A* search.

Attached are example CSV files and corresponding example text files containing the optimal path, cost of the optimal path, and number of explored nodes. Note that your function should not write anything to file. These examples are provided in separate files for convenience. Also attached is skeleton code indicating the format your implementation should take.

Note that for some examples there may be multiple correct solutions. The number of explored nodes may also be different depending on the heuristic you choose. All correct solutions will be accepted.

Example CSV file:

```
S, 0, 3
0, 0, 0
2, X, G
```

Example first output (i.e. optimal path):

```
[(0, 0), (1, 0), (2, 0), (1, 0), (1, 1), (1, 2), (0, 2), (1, 2), (2, 2)]
```

Example second output (i.e. cost of optimal path):

```
8
```

Example third output (i.e. number of states explored):

```
24
```

Grading The implementation will be worth 70 marks.

50 marks will be allocated to correctness on a series of test cases, with consideration to each of the three outputs (i.e. optimal path, optimal path cost, and number of states explored). These test cases will be run automatically by calling your implementation from another Python script. To facilitate this, your implementation must adhere exactly to the specifications.

20 marks will be allocated to human-based review of code. This human-based review will consider both correctness and use of good programming practices.

3 Questions Section

Please answer the following questions. For all questions, explain why your answers are correct.

1. What type of agent have you implemented (simple reflex agent, model-based reflex agent, goal-based agent, or utility-based agent)? [3 marks]
2. Is the task environment: [7 marks]
 - (a) Fully or partially observable?
 - (b) Single or multiple agent?
 - (c) Deterministic or stochastic?

- (d) Episodic or sequential?
 - (e) Static or dynamic?
 - (f) Discrete or continuous?
 - (g) Known or unknown?
3. What heuristic did you use for A* search for this environment? Show that your heuristic is consistent. [8 marks]
 4. Suggest a particular instance of this problem (i.e. grid) where A* search using your heuristic would find the optimal solution faster than uniform cost search. [4 marks]
 5. Suggest a particular instance of this problem (i.e. grid) where a greedy heuristic search using your heuristic would not find the optimal solution. [4 marks]
 6. Consider a modification of this problem where your agent need only pick up treasures with value totalling at least 1. Determine whether your heuristic is still consistent. [4 marks]

Grading The PDF with answers to the Questions Section will be worth 30 marks, allocated as described above.